“DjangoBlog”开源软件系统的泛读报告

组号：17
小组成员：孙子杰、何动、唐磊、唐文龙、吴政、程光文

本报告以 DjangoBlog 开源博客系统为阅读对象。系统基于 Python 与 Django 框架实现，采用 MTV 分层结构，包含文章、分类、标签、评论、用户、第三方登录与后台管理等模块。报告第一部分给出功能描述，包括用例图、用例说明、数据流图与数据字典以及核心用例的顺序图；第二部分给出界面分析；第三部分给出数据模型分析与类图；第四部分说明软件架构、各个包与主要类的作用；第五部分列出功能与类的对应关系；最后总结小组前一阶段的工作，并回答评论审核与单篇文章关闭评论两个问题。

开源软件的功能描述

DjangoBlog 是一个多用户博客系统，面向访客、注册用户和管理员三类使用者。访客可以浏览文章列表与详情、按分类或标签归档浏览、查看时间归档、使用全文检索；注册用户在此基础上可以发表评论、回复评论并对评论做出表情回应；管理员通过后台管理文章、分类、标签、评论与站点配置。系统还提供 RSS 订阅、站点地图、健康检查接口与缓存清理能力，并以插件方式扩展阅读量统计、阅读时长估算、相关文章推荐、版权声明与 SEO 优化等功能。

当前演示数据包含 20 篇文章：父分类“项目开发”下的子分类“Django 博客”集中了 19 篇说明性文章，另有 1 篇早期文章，配套 19 个标签（Django、模板渲染、ORM、数据库设计等）。首页按发布时间倒序展示文章卡片，右侧依次为两条公告卡片、热门文章（前三名使用金、银、铜名次牌）与八色轮换的标签云。

用例图和用例说明

系统的用例图如图 1 所示。图中共有四个执行者：访客（Visitor）、注册用户（Member）、管理员（Administrator）与定时任务（Timer）。注册用户与访客之间是泛化关系，表示注册用户继承了访客的全部用例。访客关联浏览文章列表、查看文章详情、按分类/标签归档浏览、查看时间归档、检索文章、用户注册与用户登录；注册用户关联发表评论、回复评论与表情回应；管理员关联文章管理、分类与标签管理、评论管理与站点设置；定时任务负责清

理缓存。发表评论、回复评论与表情回应三个用例都通过 «包含» 关系指向用户登录，表示这三项操作都必须先完成登录。

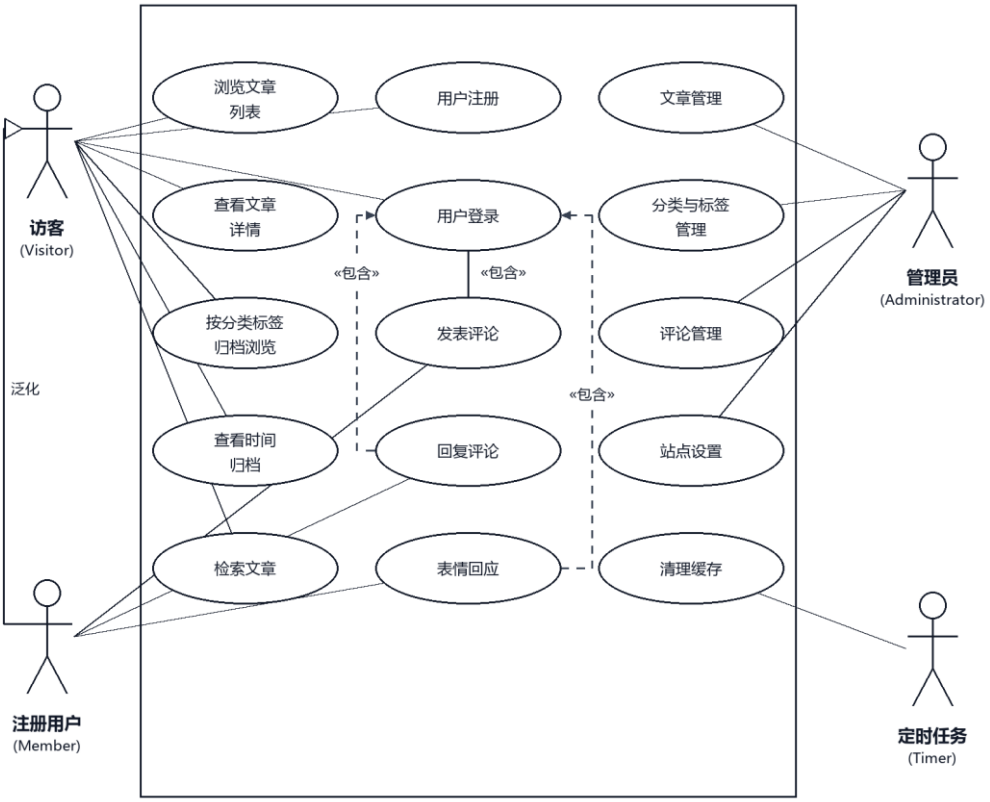


图 1 “DjangoBlog 博客系统”的用例模型

表 1 用例说明：查看文章详情

项目	内容
用例编号	UC-01
用例名称	查看文章详情
主要执行者	访客
用例目标	打开一篇文章，阅读正文并进入相关文章或评论区
范围	“DjangoBlog 博客系统”
前置条件	文章已发布（status = p），且类型为普通文章（type = a）

项目	内容
后置条件	文章内容与关联信息显示成功，该文章的阅读量加一
基本事件流	1 用户在列表页、归档页或搜索结果中点击文章标题；2 系统按“年/月/日/编号”定位目标文章；3 系统校验文章是否已发布；4 系统组装正文、标签、上一篇/下一篇与相关推荐数据；5 系统渲染文章详情页并显示评论区；6 阅读量统计插件把该文章的阅读量加一
备选事件流	1a 用户直接输入网址访问，从步骤 2 开始；2a 文章不存在，显示 404 页面；3a 文章为草稿且访问者不是作者，提示无权访问；4a 没有上一篇或下一篇，只显示其中一侧导航卡片

表 2　用例说明：检索文章

项目	内容
用例编号	UC-02
用例名称	检索文章
主要执行者	访客
用例目标	按关键词检索标题或正文，快速定位相关文章
范围	“DjangoBlog 博客系统”
前置条件	全文索引已建立（默认 Whoosh，可选 Elasticsearch）
后置条件	页面显示命中结果，命中关键词在标题与摘要中高亮
基本事件流	1 用户在导航栏搜索框输入关键词并回车；2 浏览器请求 /search/?q=关键词；3 搜索视图通过 Haystack 构建查询并开启高亮；4 搜索引擎在索引中匹配标题与正文；5 视图对结果分页并渲染 search.html；6 页面显示结果列表与关键词高亮
备选事件流	1a 关键词为空，提示输入关键词；4a 没有命中结果，显示空状态卡片并建议更换关键词；4b 索引服务不可用，返回错误提示，文章浏览功能不受影响

表 3　用例说明：用户登录

项目	内容
用例编号	UC-03
用例名称	用户登录
主要执行者	访客
用例目标	验证用户身份并建立会话，获得评论等操作权限
范围	“DjangoBlog 博客系统”
前置条件	用户已注册且账号处于启用状态
后置条件	建立登录会话，跳转到首页或登录前访问的页面
基本事件流	1 用户打开 /login/ 登录页；2 输入用户名与密码并提交；3 登录表单校验必填项；4 认证后端按用户名或邮箱查询用户并校验密码；5 校验通过后写入会话；6 页面跳转到首页或 next 指定的原页面
备选事件流	3a 字段为空，提示补充填写；4a 账号或密码错误，提示“用户名或密码错误”；4b 账号被禁用，提示账号不可用；4c 用户连续失败，可要求重新输入

表 4　用例说明：发表评论

项目	内容
用例编号	UC-04
用例名称	发表评论
主要执行者	注册用户
用例目标	在文章下发表评论，与其他读者交流
范围	“DjangoBlog 博客系统”
前置条件	用户已登录，且该文章 comment_status = o（允许评论）
后置条件	评论写入数据库；按站点审核开关决定立即显示或等待审核

项目	内容
基本事件流	1 用户打开文章详情页；2 系统显示评论列表与评论输入框；3 用户输入内容并提交；4 系统校验登录状态与正文（非空且不超过 300 字）；5 系统读取 comment_need_review 开关；6 系统保存评论（is_enable 由审核开关决定）；7 页面跳转回文章评论区并显示新评论
备选事件流	3a 未登录，跳转登录页，登录后返回本文；4a 正文为空或超长，提示重新输入；4b 文章已关闭评论，拒绝提交并提示；5a 开启审核，评论保存但不立即显示，由管理员审核后启用

数据流图和数据字典

系统的顶层数据流图如图 2 所示。外部实体有访客、注册用户、管理员与搜索引擎/订阅端。访客向系统发出浏览与检索请求，系统返回文章页面与搜索结果；注册用户提交登录凭证与评论内容，系统返回会话状态与评论结果；管理员提交内容与配置维护请求，系统返回管理结果与统计；搜索引擎与订阅端发起抓取请求，系统返回 sitemap.xml 与 RSS。

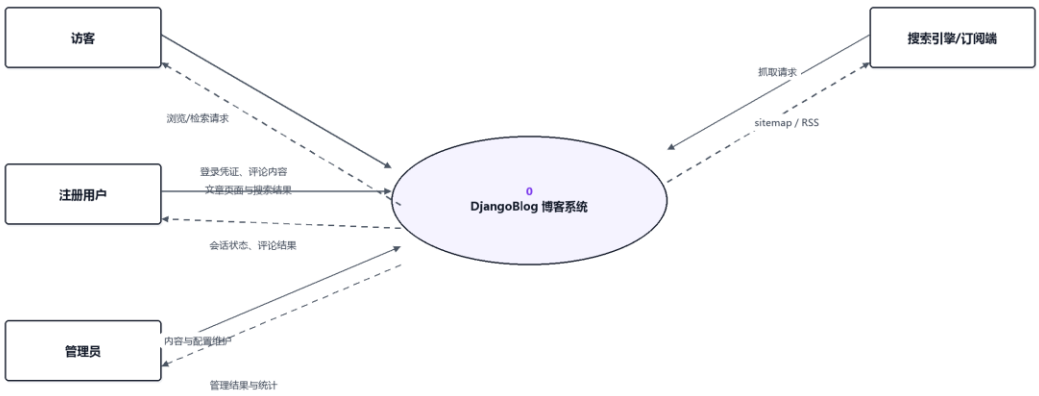


图 2　DjangoBlog 系统顶层数据流图（第 0 层）

第 1 层数据流图如图 3 所示，把系统分解为六个处理过程：P1 浏览与展示负责列表、详情与归档页面；P2 检索文章负责关键词查询；P3 用户认证负责登录、注册与第三方登录；P4 评论互动负责发表评论、回复与表情回应；P5 内容与站点管理负责后台维护；P6 站点输出与统计负责 RSS、站点地图、阅读量与缓存。数据存储包括 D1 文章/分类/标签、D2 评论与表情反应、D3 用户与第三方账号、D4 全文搜索索引、D5 缓存/统计/媒体、D6 站点配置与侧边栏。图中带 * 的“访客/注册用户”为重复出现的外部实体，用于减少长连线。

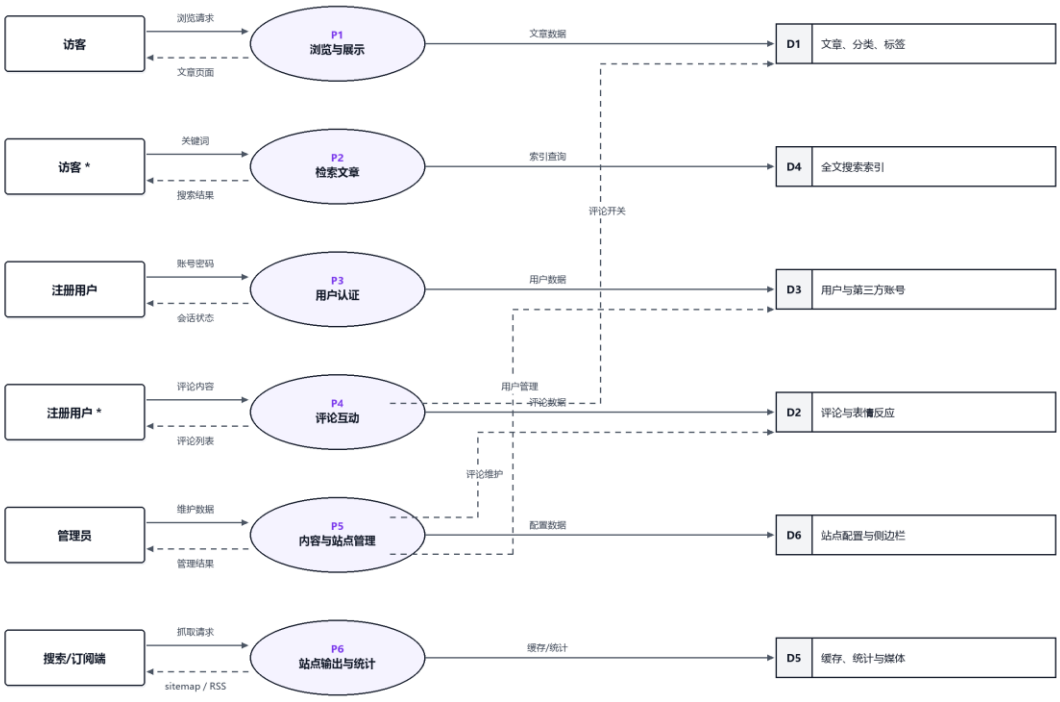


图 3　DjangoBlog 系统第 1 层数据流图

主要数据条目的数据字典如下。字段类型与约束取自项目的模型定义：标题与名称字段唯一，评论正文限制 300 字，评论表情对同一用户、同一条评论、同一种表情建立唯一约束，并为评论列表与侧边栏查询建立了组合索引。

表 5　数据字典：文章（Article）

字段	类型	约束	说明
id	AutoField	主键	文章编号
title	Char(200)	非空、唯一	文章标题
body	TextField	非空	正文（Markdown 渲染）
pub_time	DateTime	非空	发布时间，用于排序与归档
status	Char(1)	d 草稿 / p 发布	发布状态，草稿仅作者与管理员可见
comment_status	Char(1)	o 打开 / c 关闭	单篇文章的评论开关
type	Char(1)	a 文章 / p 页面	内容类型

字段	类型	约束	说明
views	PositiveInteger	默认 0	阅读量，由统计插件累加
author	FK(BlogUser)	非空	作者，一篇文章属于一个用户
category	FK(Category)	非空	所属分类
tags	M2M(Tag)	可空	标签，多对多关系

表 6　数据字典：分类与标签

字段	类型	约束	说明
Category.name	Char(30)	非空、唯一	分类名称（如“项目开发”）
Category.parent_category	FK(Category)	可空	父分类，自关联形成分类树
Category.slug	Slug(60)	可空	用于生成友好 URL
Category.index	Integer	默认 0	展示顺序
Tag.name	Char(30)	非空、唯一	标签名称（如 Django、ORM）
Tag.slug	Slug(60)	可空	标签 URL 片段

表 7　数据字典：评论与表情反应

字段	类型	约束	说明
Comment.body	Text(300)	非空、≤300 字	评论正文
Comment.author	FK(BlogUser)	非空	评论作者
Comment.article	FK(Article)	非空	被评论的文章
Comment.parent_comment	FK(Comment)	可空	父评论，自关联实现多级回复
Comment.is_enable	Boolean	默认 False	审核/展示开关，仅启用评论对外可见

字段	类型	约束	说明
Reaction.comment	FK(Comment)	非空	被反应的评论
Reaction.user	FK(BlogUser)	非空	发起反应的用户
Reaction.reaction_type	Char(10)	五种表情	👍 👎 ❤️ 😄 🎉
Reaction 唯一约束	唯一索引	comment+user+type	同一用户对同一评论同种表情只能一次

表 8　数据字典：用户（BlogUser）

字段	类型	约束	说明
username	Char(150)	非空、唯一	登录名
password	Char(128)	非空	密码哈希（Django 认证）
email	Char(254)	可空	邮箱，可用于登录与找回密码
nickname	Char(100)	可空	页面展示昵称
is_superuser	Boolean	默认 False	是否为超级管理员
is_active	Boolean	默认 True	账号是否启用
source	Char(100)	可空	注册来源（本地或第三方）

表 9　数据字典：配置与内容类

字段	类型	约束	说明
BlogSettings.site_name	Char(200)	单例配置	站点名称
BlogSettings.open_site_comment	Boolean	默认 True	全站评论总开关
BlogSettings.comment_need_review	Boolean	默认 False	新评论是否需要审核
BlogSettings.color_scheme	Char(20)	6 种主题	紫色/蓝色/绿色/橙色/粉色/红色

字段	类型	约束	说明
SideBar.name / content	Char / Text	非空	自定义公告标题与 HTML 内容
SideBar.sequence	Integer	唯一	侧边栏展示顺序
Links.link	Char(200)	非空	友情链接地址
Links.show_type	Char(1)	i/l/p/a/s	链接展示位置

核心用例的顺序图

下面给出五个核心用例的顺序图，分别对应浏览、检索、认证、评论与后台管理五类交互。顺序图按“参与者—消息—返回”描述对象之间的调用顺序，与图 1 中的用例一一对应。

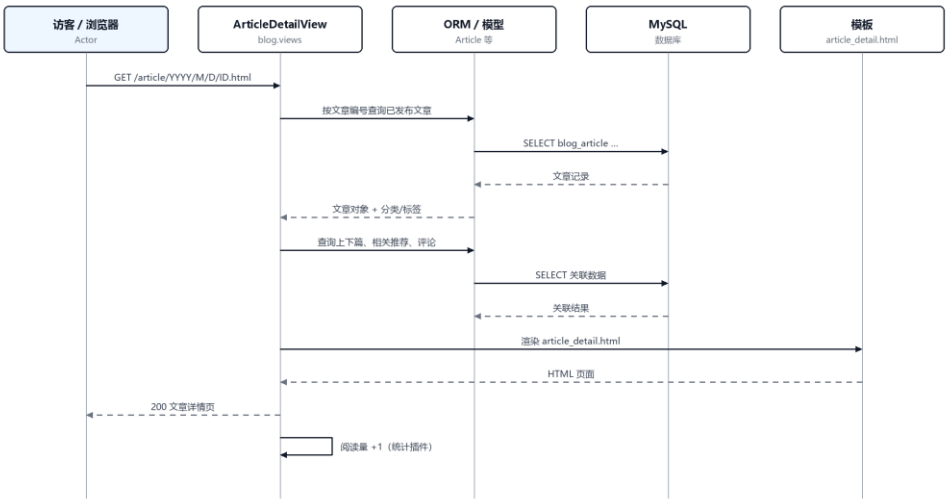


图 4　用例“查看文章详情”的顺序图

查看文章详情时，视图先按 URL 中的年月日与编号定位文章，再校验发布状态，随后读取分类、标签、上下篇与相关推荐等关联数据，最后交给模板渲染；阅读量由统计插件在请求处理后累加。

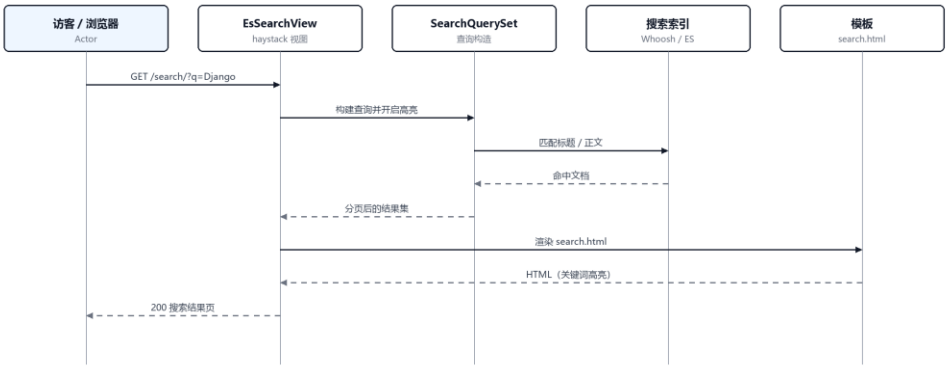


图 5　用例“检索文章”的顺序图

检索时搜索视图通过 Haystack 构造查询并开启高亮，搜索引擎在索引中匹配标题与正文，结果分页后由 search.html 渲染，命中词以主题色高亮显示。

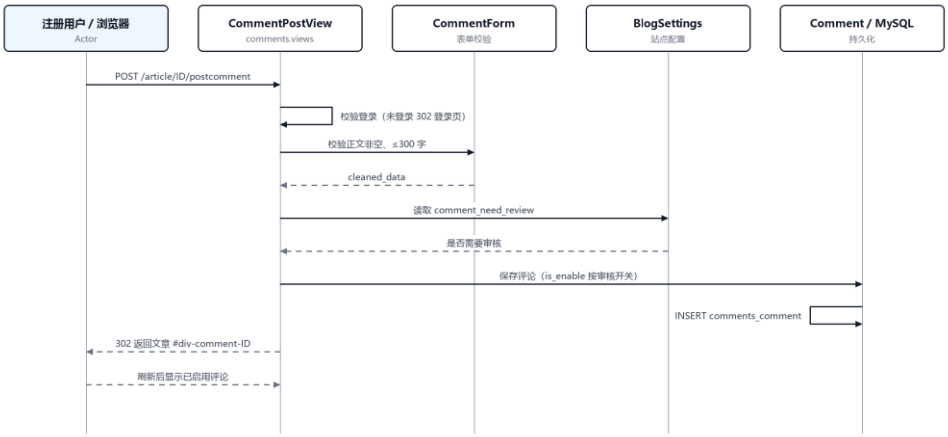


图 6　用例“发表评论”的顺序图

发表评论前先校验登录状态与正文长度，再读取站点审核开关，由此决定新评论的 is_enable 初值；保存成功后重定向回文章评论区的锚点。

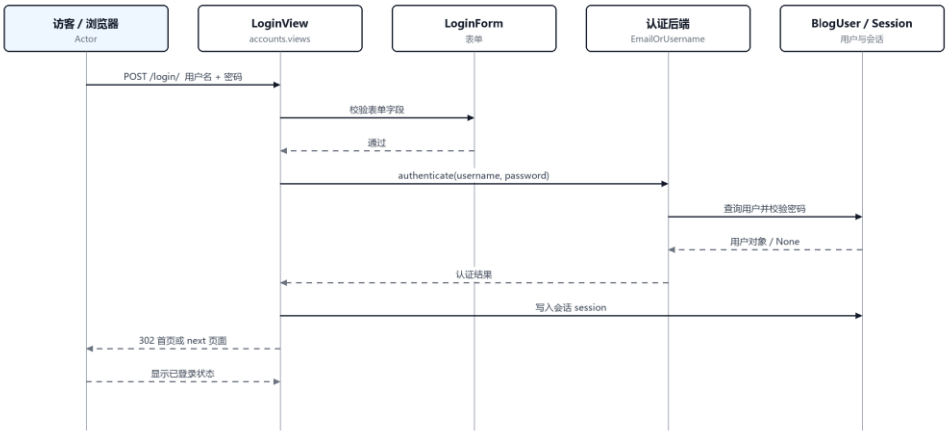


图 7　用例“用户登录”的顺序图

登录由表单校验、认证后端与会话三部分组成；认证后端支持用户名或邮箱登录，认证成功后写入会话并跳转到首页或登录前的原页面。

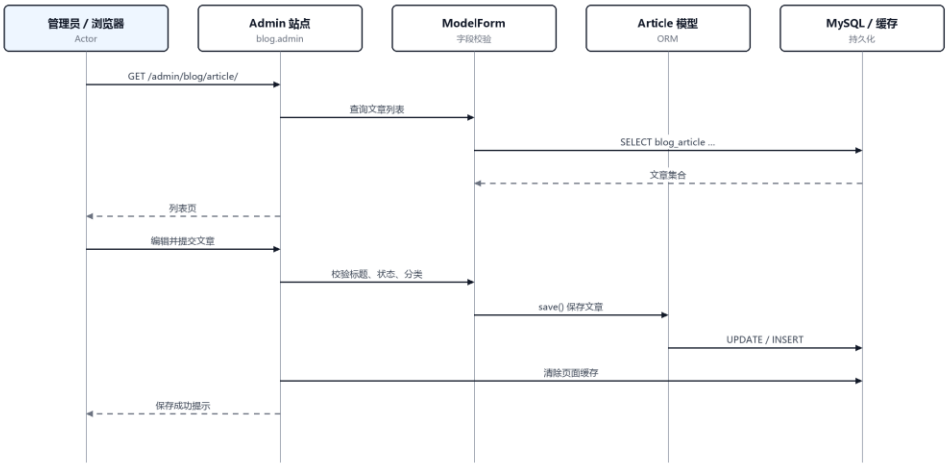


图 8　用例“文章管理”的顺序图

管理员的文章管理走 Django Admin：列表查询、表单校验与保存均由后台完成，保存后清除页面缓存，保证前台立即看到更新。

界面分析

系统界面沿用前几周界面设计的结果，采用统一的页面骨架与设计令牌。前台页面继承 share_layout/base.html，账户页面继承 base_account.html；页面容器最大宽度为 1152 像素，内容区与 300 像素宽的侧边栏组成两栏布局。颜色、圆角、边框与阴影统一定义在 frontend/src/styles/main.css 中，由 Vite 构建后输出到 blog/static/blog/dist，模板标签 vite_js

按清单文件引用。主题支持浅色与深色两种模式，并提供紫、蓝、绿、橙、粉、红六套主色。

以首页为例，内容区顶部是带主题色竖条的“近期文章”标题，下方为文章卡片列表；每张卡片展示分类、发布日期、标题、摘要、标签、阅读量与评论数。侧边栏自上而下依次是公告卡片、热门文章、标签云、近期文章与工具入口，其中热门文章的前三名使用金、银、铜三种名次牌，标签云按 8n+k 规则循环使用八种配色。文章详情页由面包屑、标题与元信息、正文、版权声明、标签、上下篇导航、相关推荐和评论区组成，整体视觉与列表页保持一致。

交互方面，顶部导航提供分类下拉、归档入口与全局搜索；搜索框回车即可提交检索；主题按钮一键切换深浅色；文章卡片与标签均可点击进入对应归档；评论区在未登录时显示登录入口，登录后显示输入框与表情回应。界面细节与完整截图见第 5 周的《软件界面设计说明书》，这里引用其中两张代表性界面作为分析依据。

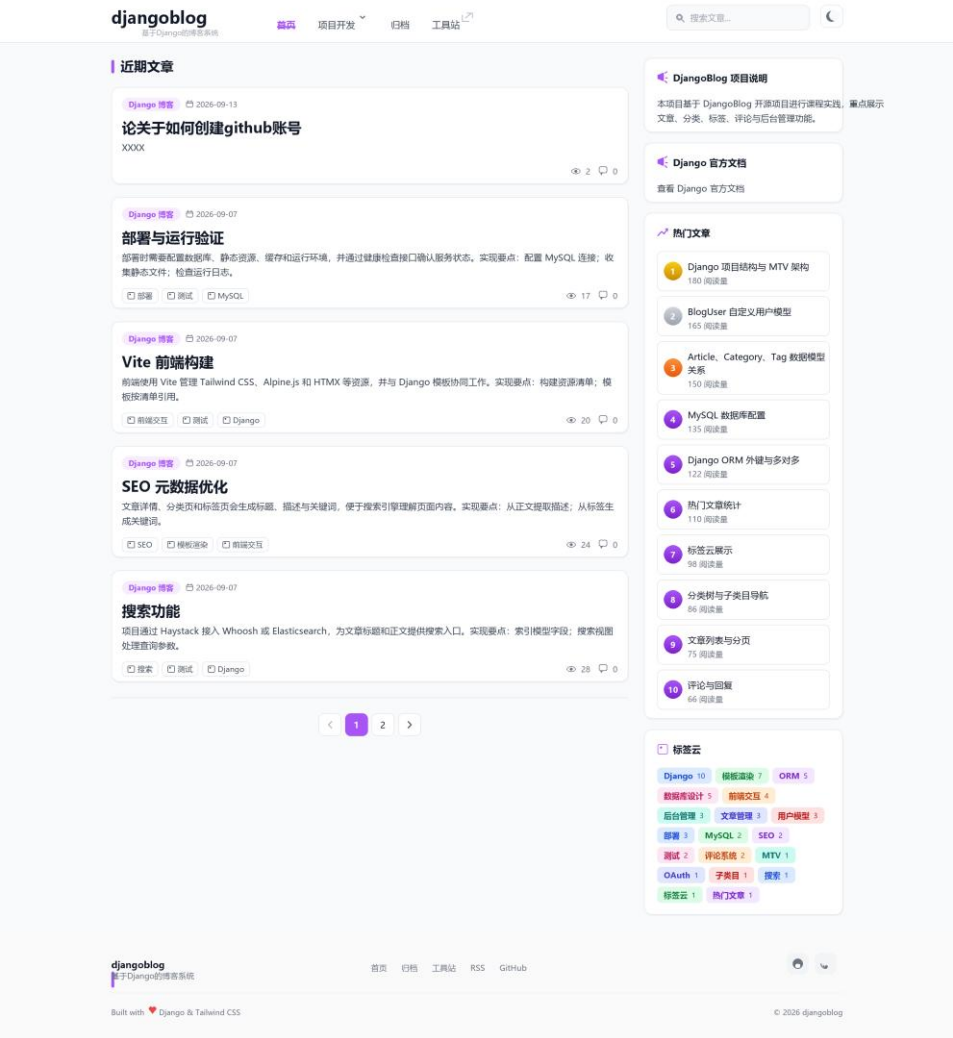


图 9　首页界面

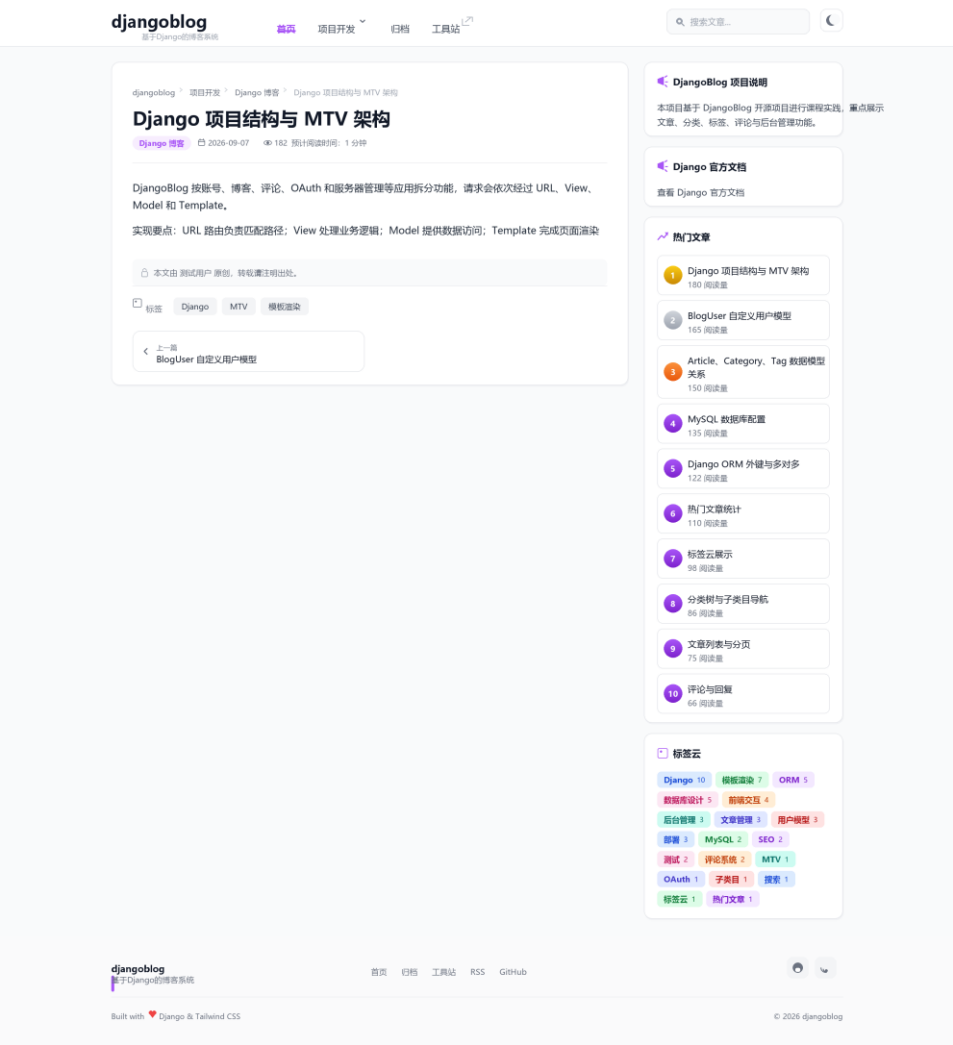


图 10　文章详情界面

数据模型分析

系统的数据模型由 5 个应用中的 11 个主要模型组成：BlogUser 是统一用户模型，Article、Category、Tag、Links、SideBar、BlogSettings 属于博客业务，Comment 与 CommentReaction 负责评论互动，OAuthUser 与 OAuthConfig 负责第三方登录。类图如图 11 所示。

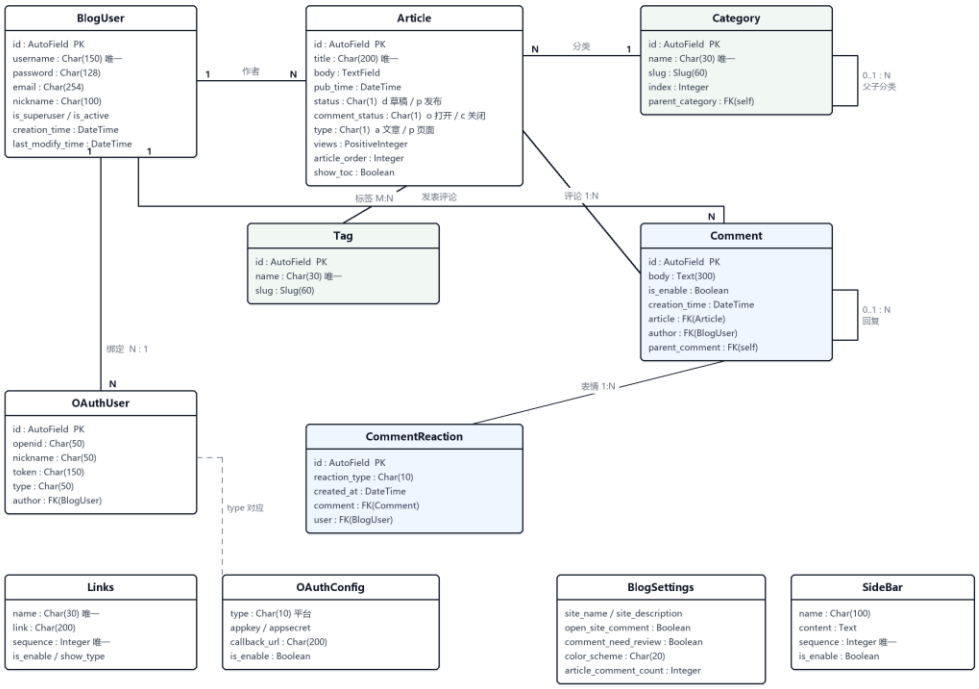


图 11　DjangoBlog 系统数据模型类图（ER 关系）

模型之间的关系可以归纳为：用户与文章是一对多（作者），分类与文章是一对多，文章与标签是多对多（Django 自动生成中间表）；文章与评论是一对多，评论通过 parent_comment 自关联形成多级回复；评论与表情反应是一对多，同一用户对同一条评论的同一种表情只能记录一次。分类通过 parent_category 自关联形成树形结构，从而实现“父类目—子类目”导航。BlogSettings 属于单例配置，模型的 clean() 方法限制系统中只能存在一条配置记录。

表 10　主要实体之间的关系

关系	基数	说明
BlogUser — Article	1 : N	一个用户可以发表多篇文章，一篇文章只有一个作者
Category — Article	1 : N	一个分类包含多篇文章，一篇文章归属一个分类
Category — Category	0..1 : N	父分类与子分类的自关联
Article — Tag	M : N	一篇文章可有多个标签，一个标签可标注多篇文章

关系	基数	说明
Article — Comment	1 : N	文章删除时评论级联删除
Comment — Comment	0..1 : N	父评论与回复的自关联
Comment — CommentReaction	1 : N	一条评论可以有多种表情反应
BlogUser — Comment / Reaction	1 : N	评论与反应都记录发起用户
BlogUser — OAuthUser	1 : N	一个站内用户可绑定多个第三方账号

约束与索引方面，Article.title、Category.name、Tag.name 均设有唯一约束；CommentReaction 对 (comment, user, reaction_type) 建有唯一索引，防止重复反应；Comment 建有 (article, parent_comment, is_enable) 与 (is_enable, -id) 两个组合索引，分别优化文章评论列表与侧边栏最新评论查询。

开源软件的软件架构及各个包和类的作用

系统采用 Django 的 MTV 架构，并在其上叠加了中间件、插件与前端构建三条扩展线。整体分为五层：客户端/表现层负责页面结构与交互；接入层负责请求进入、中间件处理与 URL 分发；业务应用层按领域拆分为 blog、accounts、comments、oauth、servermanager 五个应用，并以插件形式提供可插拔能力；数据访问层由 ORM、缓存封装与搜索封装组成；数据层包括 MySQL、缓存、搜索索引与静态/媒体资源。架构包图如图 12 所示。

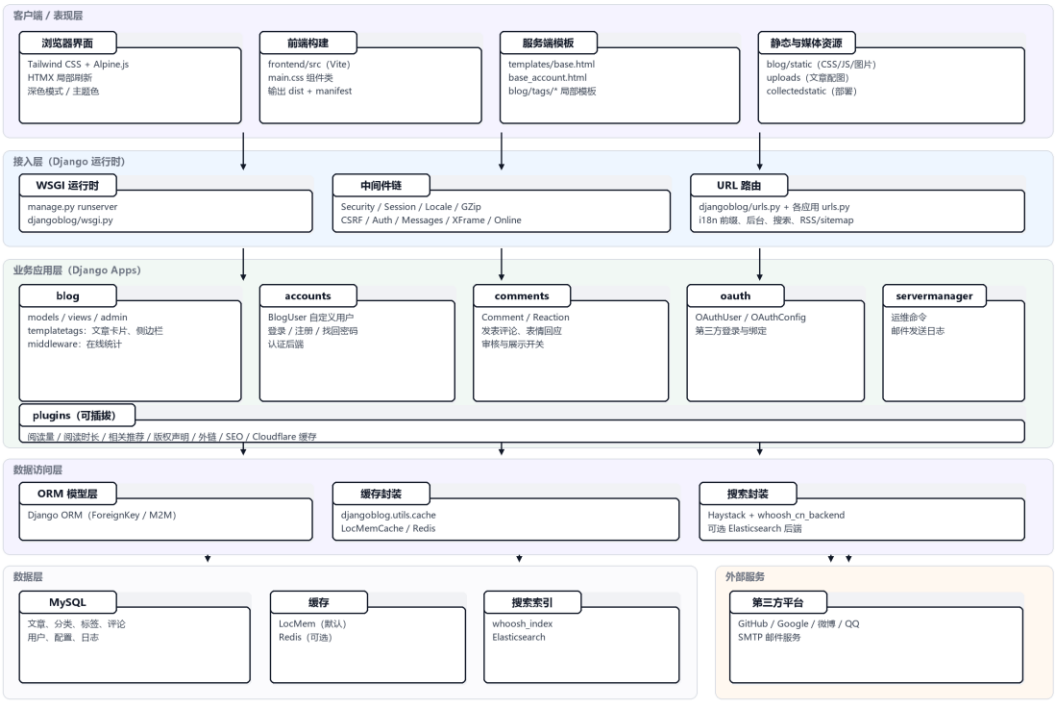


图 12　DjangoBlog 系统架构包图（分层）

请求进入后先经过安全、会话、语言、压缩、CSRF、认证与在线统计等中间件，再由 djangoblog/urls.py 分发到各应用的视图；视图通过模型访问 MySQL，或从本地缓存/Redis 读取列表数据，检索请求则交给 Haystack 访问 Whoosh 或 Elasticsearch。插件通过 djangoblog/plugin_manage 在应用启动时被加载，在文章页的指定位置挂载版权声明、相关推荐与阅读量统计等界面部件。前端资源由 Vite 构建，模板通过 vite_tags 读取清单文件引用带哈希的 CSS 与 JS。

表 11　主要包与类的作用

包	主要类 / 模块	作用
blog	Article、Category、Tag、IndexView、ArticleDetailView、EsSearchView、blog_tags	文章内容展示、归档、检索与页面局部渲染
accounts	BlogUser、LoginView、RegisterView、EmailOrUsernameModelBackend	自定义用户模型、登录注册、找回密码与认证后端

包	主要类 / 模块	作用
comments	Comment、CommentReaction、CommentPostView、CommentReactionView	评论、回复、表情回应与审核展示
oauth	OAuthUser、OAuthConfig、OAuthManager	第三方登录配置、授权回调与账号绑定
servermanager	commands、EmailSendLog	运维命令说明、邮件发送日志与站点管理辅助
plugins	各插件 plugin.py 与钩子	阅读量、阅读时长、相关推荐、版权、外链、SEO、缓存
djangoblog	settings、urls、middleware、plugin_manage、utils	项目配置、路由、中间件、插件加载与缓存工具
templates / frontend	base.html、base_account.html、main.css、main.js	页面骨架、局部模板与前端样式脚本构建

软件功能与类间的对应关系

表 12 按功能列出前台入口、处理视图、使用的模型与渲染模板，便于把界面功能与代码对象对应起来。

表 12　软件功能与类间的对应关系

功能	入口	视图类 / 函数	主要模型	模板
浏览近期文章	/	IndexView	Article、Category、Tag	blog/article_index.html
查看文章详情	/article/…html	ArticleDetailView	Article、Tag、Comment	blog/article_detail.html

功能	入口	视图类 / 函数	主要模型	模板
分类归档	/category/…html	CategoryDetailView	Category、Article	blog/article_index.html
标签归档	/tag/…html	TagDetailView	Tag、Article	blog/article_index.html
作者归档	/author/…html	AuthorDetailView	BlogUser、Article	blog/article_index.html
时间归档	/archives.html	ArchivesView	Article	blog/article_archives.html
全文检索	/search/?q=	EsSearchView	Article（索引文档）	search/search.html
用户注册	/register/	RegisterView	BlogUser	account/registration_form.html
用户登录	/login/	LoginView	BlogUser	account/login.html
发表评论	/article/ID/postcomment	CommentPostView	Comment、Article、BlogSettings	blog/article_detail.html
表情回应	/comment/ID/react	CommentReactionView	CommentReaction	评论局部模板（JSON）
后台内容管理	/admin/	Django Admin	全部业务模型	admin/*
RSS 与站点地图	/feed/、/sitemap.xml	DjangoBlogFeed、Sitemap	Article、Category、Tag	feed/sitemap 输出

功能	入口	视图类 / 函数	主要模型	模板
健康检查与清缓存	/health/、/clean	health_check、clean_cache_view	—	JSON / 重定向

总结

前一阶段小组完成了选题、需求分析、数据模型设计、界面实现与代码整理。本周围绕“读源码、建模型”展开：一方面阅读了项目的模型、视图、路由、模板与插件机制，梳理出文章、分类、标签、评论、用户与配置之间的关系；另一方面把分散的理解落实为文档，补齐了用例图、用例说明、数据流图、数据字典、顺序图、类图与架构包图。小组成员按照模块分工，一人负责业务模型与数据流，一人负责界面与模板，一人负责架构与构建流程，最后统一校对图号和术语。

这一阶段的主要收获是理解了 MTV 分层与“包—类—模板”的对应关系：视图只做数据组装，模板通过自定义标签复用局部界面，模型负责关系与约束。同时体会到工程化细节的重要性：前端样式需要构建后才会生效，数据库连接参数应优先读取环境变量，缓存与搜索索引需要单独维护。遇到的问题主要是开发环境差异（MySQL 服务、端口占用、日志目录权限）、前端构建产物与源码不同步，以及演示数据与迁移脚本的管理不规范；下一步计划是把运行环境与依赖写入统一的说明文档，补充自动化测试与持续集成，并在每次合并代码后重新构建前端与验证页面。

关于老师提出的两个问题：系统有评论审核功能。站点配置中的 comment_need_review 控制是否需要审核：开启后新评论以 is_enable=False 保存，前台只显示已启用的评论，管理员在后台启用后才会公开；关闭时评论提交后立即启用。系统也可以关闭某一篇文章的评论：文章的 comment_status 取值为 o（打开）或 c（关闭），关闭后详情页不渲染评论输入框，评论提交视图也会拒绝请求并提示“该文章评论已关闭”。此外还有 open_site_comment 作为全站评论总开关，可与单篇开关配合使用。